# ZNOTES.ORG

ALIGNED WITH THE 2026-2026 SYLLABUS

## CAIE A2 LEVEL

# COMPUTER SCIENCE (9618)

PRACTICAL

*Authorized for personal use only by Faiq Ali at  generated on 06/04/26*

# 1. Computational thinking and Problem-solving

## 1.1. Algorithms

### Linear Search

```
def linear_search(arr, target):
    # Traverse through all elements in the array
    for index in range(len(arr)):
        # If the element is found, return its index
        if arr[index] == target:
            return index
    # If the element is not found, return -1
    return -1
```

### Binary Search

- Can only be used if the array is already sorted.
- While it is true that as we add more items to the array the time taken increases, but with each iteration of the loop the data set is halved. So, the time increases by smaller and smaller increments until it reaches a constant value.

```
def binary_search(arr, x):
    low = 0
    high = len(arr) - 1
    mid = 0

    while low <= high:

        mid = (high + low) // 2

        # If x is greater, ignore left half
        if arr[mid] < x:
            low = mid + 1

        # If x is smaller, ignore right half
        elif arr[mid] > x:
            high = mid - 1

        # means x is present at mid
        else:
            return mid

    # If we reach here, then the element was not present
    return -1
```

### Bubble Sort

```
def bubble_sort(array):

        passes = 1
        swapped = True

         # If zero swaps occur in a loop, then it's already sorted
        while swapped == True:
        swapped = False
                 # Iterate through the list n times, ignoring last items as they have already been sorted
        for i in range(len(array) - passes):
                        # Swap elements if they are in the wrong order
                if array[i+1] > array[i]:
                temp = array[i+1]
                    array[i+1] = array[i]
                array[i] = temp
                # Mark that a swap has occurred
                    swapped = True
        passes += 1
```

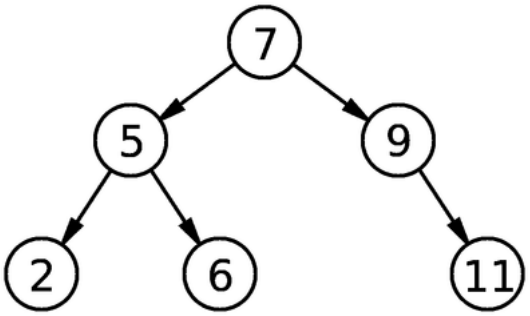
### Insertion Sort

```
def insertionSort(arr):
    n = len(arr)  # Get the length of the array

    if n <= 1:
        return  # If the array has 0 or 1 element, it is already sorted, so return

    for i in range(1, n):  # Iterate over the array starting from the second element
        key = arr[i]  # Store the current element as the key to be inserted in the right position
        j = i-1
        while j >= 0 and key < arr[j]:  # Move elements greater than key one position ahead
            arr[j+1] = arr[j]  # Shift elements to the right
            j -= 1
        arr[j+1] = key  # Insert the key in the correct position
```

## 1.2. Abstract Data Types (ADTs)

A collection of data with associated operations like a stack, queue, linked list, binary tree and graphs.

### Binary Tree




| Index | Left Pointer | Data | Right Pointer |
|---|---|---|---|
| 0 | 1 | 7 | 2 |
| 1 | 3 | 5 | 4 |
| 2 | Ø | 9 | 5 |
| 3 | Ø | 2 | Ø |
| 4 | Ø | 6 | Ø |
| 5 | Ø | 11 | Ø |

- Each parent node contains the data being stored and has 0-2 children.
- Upper most node is referred to as the root.
- A node with no children is called a leaf.
- A node with children is called a branch.
- When traversing, start at the root. When the target is lesser than the current value we move left and vice versa.
- Ø is used as the null pointer.

### Creation (Pseudocode)

```
TYPE Node
        DECLARE Left: INTEGER
        DECLARE Data: INTEGER
        DECLARE Right: INTEGER
ENDTYPE
DECLARE Tree Array[0:5] OF Node
CONSTANT NullPtr = -1 // 1 less than the lower bound of the array
// Initializing a tree
FreePtr ← 0 // Starting position of free list
RootPtr ← NullPtr // Always points to the root of the tree
FOR I ← 0 TO 4 // Links all nodes
        Tree[I].Left ← I + 1
        Tree[I].Right ← NullPtr
NEXT I
Tree[5].Left ← Null
```

### Adding an Item (Pseudocode)

```
Data ← // Some value
IF FreePtr != NullPtr THEN
        NewNodePtr ← FreePtr
        FreePtr = Tree[FreePtr].Left
        Tree[NewNodePtr].Data = Data
        Tree[NewNodePtr].Left = NullPtr
        IF RootPtr = NullPtr THEN // If there is no root node, then make th
                RootPtr = NewNodePtr
        ELSE
                CurrentPtr ← RootPtr // Starts at root
                PreviousPtr ← CurrentPtr
                WHILE CurrentPtr != NullPtr // Search, starting at the root
                        PreviousPtr ← CurrentPtr
                        IF Tree[CurrentPtr].Data > Data THEN
                                TurnedLeft ← FALSE
                                CurrentPtr = Tree[CurrentPtr].Left
                        ELSE
                                TurnedLeft = FALSE
                                CurrentPtr = Tree[CurrentPtr].Right
                        ENDIF
                        IF TurnedLeft = TRUE THEN
                                Tree[PreviousPtr].Left = NewNodePtr
                        ELSE
                                Tree[PreviousPtr].Right = NewNodePtr
                        ENDIF
                ENDWHILE
        ENDIF
ELSE
```



Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by Faiq Ali at Enderun Colleges on 06/04/26.

```
        OUTPUT "Tree is full"
ENDIF
```

#### Finding an Item (Pseudocode)

```
Target ← // Some value
CurrentPtr ← RootPtr
WHILE CurrentPtr != NullPtr
        IF Tree[CurrentPtr].Data == Target THEN
                OUTPUT "Found at ", CurrentPtr
        ELSE IF Tree[CurrentPtr].Data < Target THEN
                CurrentPtr = Tree[CurrentPtr].Left
        ELSE
                CurrentPtr = Tree[CurrentPtr].Right
        ENDIF
ENDWHILE
```

### Linked List

A linked list contains a node that points to the next node and the data value.

#### Creation (Pseudocode)

```
TYPE Node
        DECLARE Data: INTEGER
        DECLARE Next: INTEGER
ENDTYPE
DECLARE List Array[0:5] OF Node
CONSTANT NullPtr = -1 // 1 less than the lower bound of the array
// Initializing a linked list
FreePtr ← 0 // Starting position of free list
StartPtr ← NullPtr // Starting positon of list
FOR I ← 0 TO 4 // Links all nodes
        List[I].Next ← I + 1
NEXT I
List[5].Next ← Null
```

#### Adding an Item (Pseudocode)

```
Data ← // Some value
IF FreePtr != NullPtr THEN
        NewNodePtr ← FreePtr
        FreePtr = List[FreePtr].Next
        List[NewNodePtr].Data = Data
        CurrentPtr ← StartPtr
        PreviousPtr ← NullPtr
        WHILE CurrentPtr != NullPtr AND List[CurrentPtr].Data < Data
                PreviosuPtr = CurrentPtr
                CurrentPtr = List[CurrentPtr].Next
        ENDWHILE
        IF PreviousPtr = NullPtr THEN
                List[NewNodePtr].Next ← StartPtr
                StartPtr = NewNodePtr
        ELSE
                List[NewNodePtr].Next = List[PreviousPtr].Next
                List[PreviousPtr].Next = NewNodePtr
        ENDIF
ELSE
        OUTPUT "List is full"
ENDIF
```

#### Finding an Item (Pseudocode)

```
Target ← // Some value
CurrentPtr ← StartPtr
Found ← FALSE
WHILE CurrentPtr != NullPtr AND NOT Found
        IF List[CurrentPtr].Data == Target THEN
                OUTPUT "Found at ", CurrentPtr
                Found = TRUE
        ELSE
                CurrentPtr = List[CurrentPtr].Next
        ENDIF
ENDWHILE
```

#### Deleting an Item (Pseudocode)

```
Target ← // Some value
CurrentPtr ← StartPtr
PreviousPtr ← NullPtr
WHILE CurrentPtr != NullPtr AND List[CurrentPtr] != Target
        PreviousPtr = CurrentPtr
        CurrentPtr = List[CurrentPtr].Next
ENDWHILE
IF CurrentPtr = NullPtr THEN
        OUTPUT Target, " does not exist"
ELSE
        IF CurrentPtr = StartPtr THEN
                StartPtr = List[StartPtr].Next
        ELSE
                List[PreviousPtr].Next = List[CurrentPtr].Next
        ENDIF
        List[CurrentPtr].Next = FreePtr
        FreePtr = CurrentPtr
ENDIF
```

### Queue

- Each element in a queue stores one data item
- Pointer for the front and the back of the queue.
- Works on the principles of "First in first out".
- Has two accessible ends.

#### Creation (Pseudocode)

```
CONSTANT NullPtr = -1 // 1 less than the lower bound of the array
CONSTANT MaxSize = 8
DECLARE Queue Array[0:MaxSize-1] OF String
DECLARE FrontPtr : INTEGER
DECLARE EndPtr : INTEGER
DECLARE Size : INTEGER
// Initializing a queue
PROCEDURE InitialiseQueue()
        FrontPtr ← NullPtr
        EndPtr ← NullPtr
        Size ← 0
ENDPROCEDURE
```

#### Adding an Item (Pseudocode)

```
PROCEDURE Enqueue(Data)
        IF Size >= MaxSize THEN
                OUTPUT "Queue overflow"
        ELSE
                EndPtr = EndPtr + 1
                IF EndPtr > MaxSize - 1 THEN
                        EndPtr = 0 // Go back to the start
                ENDIF
                Queue[EndPtr] = Data // Add item
                Size = Size + 1
        ENDIF
ENDPROCEDURE
```

#### Deleting an Item (Pseudocode)

```
FUNCTION Dequeue() RETURNS String
        DECLARE Item : String
                IF Size <= 0 THEN
                        OUTPUT "Queue underflow"
                ELSE
                        Size = Size - 1
                        Item = Queue[FrontPtr] // Get item
                        IF Size == 0 THEN
                                CALL InitialiseQueue()
                        ELSE
                                FrontPtr = FrontPtr + 1
                                IF FrontPtr > MaxSize - 1 THEN
                                        FrontPtr = 0 // Go back to start
                                ENDIF
                        ENDIF
                ENDIF
        RETURN Item
ENDFUNCTION
```

### Stack

- Each element stores one data item.
- Pointer to the current place of the stack.
- Stores elements on the principles of "First in last out".
- Has one accessible end.
- Use cases include: recursion, interrupt handling, evaluation of RPN expressions and implementing ADTs

#### Creation (Pseudocode)

```
CONSTANT NullPtr = -1 // 1 less than the lower bound of the array
CONSTANT MaxSize = 8
DECLARE Stack Array[0:MaxSize-1] OF String
DECLARE Ptr : INTEGER
// Initializing a stack
PROCEDURE InitialiseStack()
        Ptr ← NullPtr
ENDPROCEDURE
```

#### Adding an Item (Pseudocode)

```
PROCEDURE Push(Data)
        IF Ptr >= MaxSize - 1 THEN
                OUTPUT "Stack overflow"
        ELSE
```



Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by Faiq Ali at Enderun Colleges on 06/04/26.

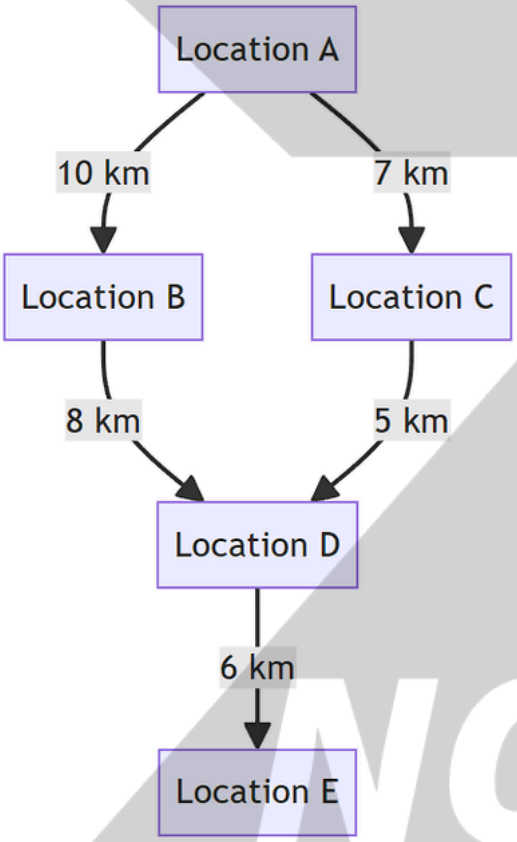
```
                Ptr = Ptr + 1
                Stack[Ptr] = Data
        ENDIF
ENDPROCEDURE
```

#### Deleting an Item (Pseudocode)

```
FUNCTION Pop() RETURNS String
        DECLARE Item : String
                IF Ptr <= NullPtr THEN
                        OUTPUT "Stack underflow"
                ELSE
                        Item = Stack[Ptr]
                        Ptr = Ptr - 1
                ENDIF
        RETURN Item
ENDFUNCTION
```

#### Graph

- A structure used to model relationships between objects.
- Consists of nodes and edges.
- Can be bidirectional or unidirectional.
- An example use for a graph includes google maps. Each location is represented by a node. The edges represent the routes between the nodes.




### 1.3. Big O Notation

Big O Notation is used to help compare algorithms that achieve the same end result based on time complexity and space complexity.

Time complexity: The increase in time taken as the size of the array increases

Space complexity: The increase in memory used as the size of the array increases.

| Notation | Example |
|---|---|
| $O(1)$ | A function that gets the first item from an array |
| $O(n)$ | Linear search |
| $O(log_2n)$ | Binary Search |
| $O(n^2)$ | Bubble/Insertion Sort |

### 1.4. Recursion

- A function/procedure that calls itself is known as a recursive function/procedure.
- Must have a base case and a general case. The general case calls the recursive function.
- Some compilers may also convert recursive functions into iterative ones.
- Advantage: Cleaner code
- Disadvantage: Large amounts of memory needed

When a recursive function is called, the address of the line as well as any additional arguments are saved to the stack. This continues until the base call is reached. Then from there it works backwards carrying out any operations until the first line is reached.

## 2. Further Programming

### 2.1. Programming Paradigms

A programming paradigm is a fundamental style of programming.

#### Low Level

Low level programming means manipulating memory directly.

```
Data Movement
LDM <#n/Bn/&n> ; Loaded to ACC
LDD <address> ; Contents of address loaded to ACC
LDI <address> ; Contents of address of address loaded to ACC
LDX <address> ; Content of IX added to address to form a  new address and c
LDR <#n/Bn/&n> ; Loaded to IX
MOV <register> ; Move contents of ACC to <register>
STO <address> ; Store contents of ACC at <address>
IN <> ; Store the ASCII value of the char in the ACC
OUT <> ; Output the character based on the ASCII value in  the ACC

Arithmetic Operations
ADD <#n/Bn/&n> ; Add to ACC
SUB <#n/Bn/&n> ; Subtracts from ACC
INC <register> ; Increment register
DEC <register> ; Decrement register

Conditional and Unconditional Jumps
JMP <address> ; Jump to given address
JPE <address> ; Jump if true after comparison
JPN <address> ; Jump if false after comparison

Comparison
CMP <#n/Bn/&n> ; Checks if ACC is equal to value
CMI <address> ; Check if contents of address of the address is equal to ACC
```

- # Represents a decimal number
- B Represents a binary number
- & Represents a hexadecimal number

Addressing Modes

- Immediate: The operand is the value used in the instruction.
- Direct: The value of the address is used in the instruction.
- Indirect: The operand stores an address that points to another address that stores the data.
- Indexed: The operand is added to the index and the data at the address is used.
- Relative: The operand is added to a set address to calculate where to read value from, offsets. It is used to allow for relocatable code as all addresses are base + offset.

#### Imperative

Imperative programming involves writing a program as a sequence of steps that is executed by the processor, like pseudocode.

#### Object Oriented

Object oriented programming is based on objects interacting with each other.

Object: An object is an instance of a class.

Properties/Attributes/Fields: The data items of the class. By setting properties to private it ensures they can only be accessed by the class and enforces encapsulation.

Method: The functions/procedures that are a part of a specific class.

Class: A data type that combines data structures and methods, which operate on that data structure, and serves as a template that is used to make objects.

Inheritance: The concept that all attributes and methods of the base class are passed on to the subclass.

Polymorphism: The concept that a method may behave differently for different classes in the hierarchy.

Aggregation/Containment: When one class contains another class within it.

Encapsulation: Encapsulation involves bundling variables (data) and methods within a class to restrict direct access to the variables.

Getter: A method to access the associated attribute.

Setter: A method to set the value of the associated attribute.

#### Declarative

Declarative programming is fact oriented and there is no need to consider sequence of execution.

#### Fact

A clause without a body



Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by Faiq Ali at Enderun Colleges on 06/04/26.

```
capitalCity(paris). /* Means Paris is a captial city */
capitalCity(paris). /* Running it a second time returns true because the database knowns Paris is a capital city */
```

**Using Variables**

```
cityInCountry(paris, france). /*Means Paris is a city in France*/
cityInCountry(City, france). /*Retuns Paris*/
```

If a fact has more arguments but we only need one of them, you can replace the ones you don't need with _ (an anonymous variable).

**Rules**

```
parent(fred, jack).
parent(fred, alia).
parent(fred, paul).
parent(dave, fred).
sibling(A,B) :- parent(P,B), parent(P,A), not(A=B). /*This is a rule that says if two kids are siblings if they share a parent*/
sibling(jack,X) /*Returns alia and paul*/
```

## 2.2. File Processing

These are the pseudocodes available for files.

```
OPENFILE name FOR WRITE // WRITE/READ/APPEND/RANDOM
WRITEFILE name, value // Writes one line
READFILE name, variable // Reads one line
GETRECORD name, identifier // Write a record to a file
PUTRECORD name, identifier // Reads a record from a file
SEEK name, address // Move to a specific address within a file
CLOSEFILE name
```

**Example**

```
OPENFILE "Text.txt" FOR READ
WHILE NOT EOF("Text.txt") DO
 READFILE "Text.txt", Value
 OUTPUT Value
ENDWHILE
CLOSEFILE "Text.txt"
```

### Sequential/Serial Files

**Reading**

```
OPENFILE CarFile FOR WRITE
FOR i ← 1 TO MaxRecords
 GETRECORD CarFile, Car[i]
NEXT i
CLOSEFILE CarFile
```

**Writing**

```
OPENFILE CarFile FOR READ
FOR i ← 1 TO MaxRecords
 PUTRECORD CarFile, Car[i]
NEXT i
CLOSEFILE CarFile
```

### Random Files

**Reading**

```
OPENFILE CarFile FOR RANDOM
Address ← Hash(ThisCar.VehicleID)
SEEK CarFile, Address
GETRECORD CarFile, ThisCar
CLOSEFILE CarFile
```

**Writing**

```
OPENFILE CarFile FOR RANDOM
Address ← Hash(ThisCar.VehicleID)
SEEK CarFile, Address
PUTRECORD CarFile, ThisCar
CLOSEFILE CarFile
```

## 2.3. Exception Handling

- Used to handle runtime errors (exceptions).
- Keeps program running despite a runtime error instead of crashing
- Traps the error and produces more useful errors

### Exception

An event that occurs during the program's executions causing it to halt. Examples:

- File not found
- Array out of bounds
- Hard disk crash
- Dividing by zero



*Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.*
*This document is authorised for personal use only by Faiq Ali at Enderun Colleges on 06/04/26.*

ZNOTES.ORG

# CAIE A2 LEVEL
# COMPUTER SCIENCE (9618)

THEORY

© ZNotes Education Ltd. & ZNotes Foundation 2026. All rights reserved.
This version was created by Faiq Ali on 06/04/26 for strictly personal use only.
These notes have been created by Talal Aamer for the 2024-2025 syllabus
The document contains images and excerpts of text from educational resources available on the internet and printed books.
If you are the owner of such media, test or visual, utilized in this document and do not accept its usage then we urge you to contact us
and we would immediately replace said media. No part of this document may be copied or re-uploaded to another website.
Under no conditions may this document be distributed under the name of false author(s) or sold for financial gain.
"ZNotes" and the ZNotes logo are trademarks of ZNotes Education Limited (registration UK00003478331).